Content-Anchored Context Windows for Late-Chunked Retrieval

Tianlei Pang (1008751307), Kent Chen (1007307200) | CSC2508 Research Proposal | November 2, 2026

Abstract

This project tests whether content-anchored encoding windows can reduce the cost of updating contextual chunk embeddings while retaining retrieval quality. The searchable chunks stay fixed; only the choice of which chunks share an encoder pass changes. The project fits Section 4.2 of the course: retrieval granularity, update cost, and retrieval-system co-design.

1 Motivation

Late Chunking encodes a document jointly and then pools token outputs into separate chunk vectors, so each chunk embedding carries document context. Long Late Chunking extends this to documents longer than the encoder window by using bounded, overlapping windows [1].

Shared context can improve retrieval, but it also creates *update dependencies*: an unchanged chunk may need a new vector because neighbouring text changed, and an insertion can additionally shift every later positional window. These effects should be separated from changes to the searchable chunks themselves, and this project measures them separately.

2 Research Questions

RQ1: With the same encoder and the same retrieval chunks, how do document-wide, positional-window, and local contextual encoding compare in retrieval quality and exact refresh cost?

RQ2: Can content-anchored context windows improve this trade-off over tuned positional windows and inexpensive metadata-enriched independent encoding, and how close do they come to the update locality of stateful grouping without storing grouping state?

3 Related Work and Contribution

ConTEB and InSeNT provide a peer-reviewed benchmark for context-sensitive retrieval and a trained contextual encoder [2]. Recent work measures revision-induced recomputation for independently embedded fixed, recursive, and content-defined chunks [3], and static multi-objective work compares Late Chunking with title-enriched alternatives [4]. Content-defined boundaries and embedding caches also appear in Steadlith and KARA [5, 6].

Boundary drift, caching, and metadata enrichment are therefore established ideas. The contribution here is their *controlled evaluation at the context-window layer*, including the extra dependencies that contextual encoding creates, while holding the retrieval units constant.

4 Method: Content-Anchored Encoding Groups

Anchors. A document is represented as an ordered sequence of retrieval chunks. An anchor is placed after a chunk when

$$\text{hash64}(\text{chunk text}) \bmod k = 0, \qquad k \in \{4, 8\},$$

using a fixed content hash. Anchor status therefore depends only on the chunk's own text, not on its ordinal position or on any query.

Grouping. Whole chunks between anchors are packed into encoding groups subject to a token cap, and packing restarts at each anchor. After an insertion, grouping can resynchronize at the next unchanged anchor. The hard cap bounds input length but not worst-case propagation distance; anchor-free runs remain a failure case to measure.

Encoding and caching. Each group is encoded jointly and pooled back into its original chunk spans. A cache identity covers the full tokenized input, the pooling spans, prompts, and the encoder and tokenizer revisions, so a vector is reused only under identical encoding inputs. A changed shared prefix invalidates every dependent group.

Prefix ablation. An optional variant prepends an existing title or document lead, capped at 64 tokens, to each group.

The method uses frozen models and requires no generated summaries and no model training.

5 Experimental Design

5.1 Reproduction

The first step reproduces the independent-versus-late comparison of [1], Section 4.1, using `jina-embeddings-v2-small-en` and 256-token chunks on SciFact. This verifies token spans, special tokens, normalization, and chunk-to-document ranking. Long Late Chunking is then implemented from Algorithm 2, including its overlap ownership rule.

The update study itself uses ConTEB's supplied chunks and chunk-level relevance labels. Document-level relevance scores from the reproduction are not transferred between the two tasks.

5.2 Encoding Policies Compared

All policies share the encoder, the retrieval chunks, and exact vector search; they differ only in which chunks are encoded together (Table 1).

Policy	Encoding unit	What an edit invalidates
Independent	Each chunk alone	Edited chunks only
Independent + prefix	Chunk + title/lead ($\leq$64 tokens)	Edited chunks; all if prefix changes
Document-wide LC	Whole document in one pass	The whole edited document
Positional windows (Long LC)	Overlapping fixed-size windows	Edited window + every later window
Local contextual	Chunk + bounded neighbourhood	Chunks whose neighbours changed
Sticky ID groups	Stored groups, tracked by chunk ID	Edited group (+ any split or merge)
Content-anchored (proposed)	Chunks between content anchors	Edited group, to next unchanged anchor

Table 1: Encoding policies (LC = Late Chunking). Retrieval chunks are identical across all policies.

Sticky ID groups. This baseline stores each chunk's group and updates it locally: an edited chunk re-encodes its own group, an inserted chunk joins its predecessor's group (splitting it at the token cap), and a deletion merges an undersized group with a neighbor. With the stable IDs our edit workload provides, it bounds achievable update locality. Unlike content anchors, however, it needs stored state and depends on edit history, so we evaluate it over edit sequences to measure fragmentation and drift from a from-scratch grouping.

5.3 Data

MLDR's training split is used for development and its test split for held-out evaluation. ConTEB COVID-QA serves as a transfer set, and Insurance as a structured case study [2]; because Insurance has only one source document, it is reported descriptively. Native chunk order is preserved by sorting numeric chunk suffixes explicitly.

A frozen InSeNT checkpoint is evaluated on a selected configuration to test whether conclusions survive a stronger contextual encoder. This does not reproduce InSeNT training.

5.4 Configuration and Tuning

Context caps of 1024, 2048, and 4096 tokens are swept, counting prefixes and special tokens. Anchor frequency and window overlap are tuned only on development data.

The analysis compares actual context-length distributions and quality–cost frontiers; smaller average groups alone do not establish an anchoring benefit. Oversized chunks are encoded alone if they fit the model limit; otherwise the document is excluded across all methods and coverage is reported. Evidence is never truncated.

5.5 Edit Workload

Edits include equal-token-length replacements with verified boundaries, insertions and deletions between retained chunks, and shared-prefix changes at early, middle, and late positions. Retained chunks keep stable IDs, which isolates context-window effects.

Cost is measured on all edits. Updated-snapshot quality is evaluated only on an audited subset whose

queries and relevance labels remain valid; unchecked semantic edits are treated as cost-only, not as a retrieval benchmark.

5.6 Metrics and Statistical Analysis

Quality. nDCG@10 and Recall@10 on the full candidate corpus.

Cost. Unique refreshed input tokens, encoder calls, rewritten vectors, and warmed refresh latency, with embedding computation separated from regrouping and vector writes. Token counts are not a substitute for measured time.

Statistics. Paired differences are bootstrapped by source document; Insurance queries are not treated as independent documents.

Decision rule. The proposed policy is compared with Long Late Chunking, with both configurations selected on development data and frozen for testing. A tolerance of 0.02 absolute nDCG (on the 0–1 scale) is declared in advance: retained quality is claimed only if the paired 95% confidence interval for the difference lies above -0.02. All controls and refresh savings are reported.

Correctness checks. Every incremental result must match a full rebuild under its own policy within numerical tolerance; for sticky ID groups, the rebuild uses the stored group map. Updates with no edits must make zero encoder calls. Full-context baselines refresh only edited documents, not the whole corpus.

6 Execution Plan and Expected Outcomes

Work begins with 20 documents and the 33M-parameter Jina encoder to profile inference, then expands to held-out MLDR and COVID-QA. Frozen InSeNT validation is limited to selected policies. Exact vector search keeps ANN approximation and production infrastructure outside the core experiment, and vectors are reused only under identical encoding inputs rather than accepting stale contextual vectors.

The hypothesis is that resynchronization and compact source context preserve useful evidence at lower refresh cost. The magnitude of quality loss, the effect of anchor deletions, and the dependence on the encoder are uncertain. If tuned positional windows or prefix-enriched independent encoding dominate, that result will be reported together with an analysis of context distance, prefix invalidation, and anchor-free runs. Neither novelty nor a performance improvement is assumed.

7 Schedule

Dates	Milestone
Nov 4–10	Reproduce baselines; verify pooling and cache behavior
Nov 11–17	Implement edits, anchored groups, and sticky ID groups
Nov 18–Dec 1	Evaluate quality and refresh cost
Dec 2–15	Validate with the stronger encoder; analyze failures
Remaining time	Write the report; release reproducible configurations

References

[1] M. Günther et al. 2024, v3 2025. Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models.

[2] M. Conti et al. 2025. Context is Gold to find the Gold Passage: Evaluating and Training Contextual Document Embeddings. EMNLP.

[3] Y. Bian et al. 2026. Chunk Recomputation under Real Document Revisions: Measuring Update Cost and Retrieval Quality of Fixed, Recursive, and Content-Defined Chunking. Preprint.

[4] L. Caspari et al. 2026. When Is Complex Chunking Worth It? A Multi-Objective Evaluation of Chunking Methods at Scale.

[5] Steadlith contributors. 2026. Content-defined chunking algorithm. Software documentation.

[6] KARA contributors. 2026. KARA toolkit for efficient document updates in RAG. Software repository.